

CPSC 440/540 Machine Learning (January-April, 2022)

Assignment 2 (due Friday February 11th at midnight)

The assignment instructions are the same as for the previous assignment, but for this assignment you can work in groups of 1-2. However, please only hand in one assignment for the group.

1. Name(s): Wira Azmoon Ahmad
2. Student ID(s): 55246318

1 Bernoulli Distributions

1.1 Inference

Consider a Bernoulli distribution with $\theta = 0.12$.

1. Suppose we generate 10 IID samples $\{x^1, x^2, \dots, x^{10}\}$ according to this model. What is the probability that all 10 samples are equal to 0?

Answer:

$$\Pr(x^1 = x^2 = \dots = x^{10} = 0) = (1 - 0.12)^{10} = 0.278500976$$

2. What is the probability that exactly one sample is equal to 1, and the other 9 are equal to 0?

Answer:

$$\Pr(1 \text{ sample equal } 1, \text{ other } 9 \text{ equal } 0) = 10 \times (1 - 0.12)^9 (0.12) = 0.3797740582$$

3. Suppose we need to base decisions on samples from this dataset, and so we want collect a set of 1000 samples. Would it make sense to chose the decoding of the 1000 samples (the “most likely” samples to be seen) to make decisions? Explain why or why not?

Answer: No. Decoding would infer that all 1000 samples will be equal to 0. Depending on the use case, making a decision based on this may not be helpful. To predict 0 COVID cases for example would not contribute to better policies.

4. Write a function that generates t IID examples from this distribution, that uses Julia’s `rand()` function as the source of IID randomness (assuming that each call to `rand()` returns a number uniformly-distributed between 0 and 1). Hand in the code.

Answer:

```
function generate(t, theta = 0.12)
    return rand(t) .<= theta
end
```

5. Consider a game where you get \$1 if a sample from this distribution is 0, and lose \$5 if the sample is 1. What is the expected \$ value you get from playing this game?

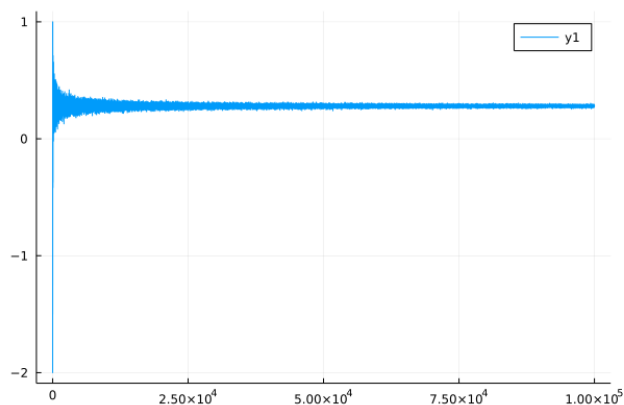
Answer:

$$E(X) = 1 \times (1 - 0.12) - 5 \times 0.12 = \$0.28$$

6. A way to approximate the answer of the previous question is given by

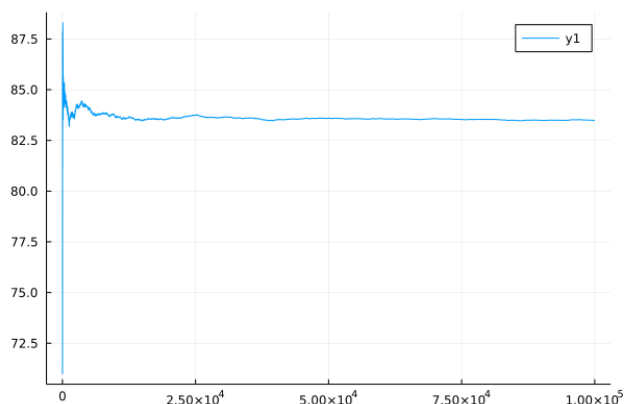
$$\frac{1}{t} \sum_{i=1}^t f(x^i),$$

where each x^i is one of t IID sample and f gives the \$ value for that sample (either 1 or -5). Implement this approximation, and hand in a plot the of value of this approximation as you vary the number of samples t from 1 to 100,000.



7. Suppose that you wanted to compute the expected number of samples from this Bernoulli that you would need to generate before seeing 10 values of 1. It possible to compute this expectation exactly, but suppose we really like sampling and we are too lazy to do the exact calculation. Give an approximation of this quantity based on generating samples, similar to the one used in Part 6.

Answer: Approximated value: 83.31604



8. Suppose that the *rand()* function was broken in the new version of Julia, in the sense that the numbers it returned were found to not look like uniform samples.¹ Also suppose that the *randn()* function, which generates samples from a standard normal distribution, worked well. Describe a way to generate samples from a Bernoulli distribution that relies on the *randn()* function instead of the *rand()* function. Hint: you may to look up the CDF (and its inverse the “quantile” function) of the standard normal distribution.

Answer: Using the quantile function Q of the normal distribution, the value obtained from *randn()* can be compared with $Q(0.12)$. A sample is equal to 1 iff

$$randn() \leq Q(0.12)$$

¹Some published research works have had to be revised due to bad random number generators.

1.2 Learning

1. Consider a dataset consisting of binary samples $\{x^1, x^2, \dots, x^n\}$. Consider the binomial likelihood for such a dataset,

$$p(x^1, x^2, \dots, x^n | \theta) = \binom{n}{k} \theta^k (1 - \theta)^{n-k},$$

where $k = \sum_{i=1}^n x^i$ is the number of 1 values in the dataset and $\binom{n}{k} = \frac{n!}{k!(n-k)!}$ is the number of combinations of k items that exist among n items. This likelihood measures the probability of seeing exactly k values of 1 in the dataset, as opposed to the Bernoulli likelihood which measures the likelihood of seeing the precise sequence $x^1, x^2, \dots, x^n$. [Derive the MAP estimate of \$\theta\$ for this model if we use a beta prior on \$\theta\$ with hyper-parameters \$\alpha\$ and \$\beta\$.](#) You can assume that k and $(n - k)$ are both positive. You can also assume that the data is independent of the hyper-parameters if we condition on the parameters, which mathematically is equivalent to $p(X | \theta, \alpha, \beta) = p(X | \theta)$.

Answer:

$$\begin{aligned} \Pr(\theta|X, \alpha, \beta) &\propto \Pr(X|\theta, \alpha, \beta) \Pr(\theta|\alpha, \beta) \\ &\propto \Pr(X|\theta) \theta^{\alpha-1} (1 - \theta)^{\beta-1} \\ &\propto \theta^k (1 - \theta)^{n-k} \theta^{\alpha-1} (1 - \theta)^{\beta-1} \\ &= \theta^{k+\alpha-1} (1 - \theta)^{n-k+\beta-1} \end{aligned} \tag{1}$$

$$\log \Pr(\theta|X, \alpha, \beta) \propto (k + \alpha - 1) \log \theta + (n - k + \beta - 1) \log(1 - \theta)$$

Finding the derivative of the expression wrt θ and equating to 0,

$$\begin{aligned} \frac{k + \alpha - 1}{\theta} - \frac{n - k + \beta - 1}{1 - \theta} &= 0 \\ (1 - \theta)(k + \alpha - 1) &= \theta(n - k + \beta - 1) \\ \theta &= \frac{k + \alpha - 1}{n + \alpha + \beta - 2} \end{aligned}$$

Since

$$\begin{aligned} \theta_{MAP} &\in \arg \max_{\theta} \Pr(X|\theta) \Pr(\theta) \\ \theta_{MAP} &= \frac{k + \alpha - 1}{n + \alpha + \beta - 2} \end{aligned}$$

2. We often use Bernoulli distributions as parts of more-complicated distributions. In these cases we often to need to maximize a weighted log-likelihood of the form

$$f(\theta) = \sum_{i=1}^n v^i \log p(x^i \mid \theta),$$

where each v^i is a non-negative weight for example i . [Derive the MLE for this model when we use a Bernoulli likelihood for \$p\(x^i \mid \theta\)\$.](#)

Answer:

$$\begin{aligned} f(\theta) &= \sum_{i=1}^n v^i (x^i \log \theta + (1 - x^i) \log(1 - \theta)) \\ \frac{df}{d\theta} &= \sum_{i=1}^n v^i \left(\frac{x^i}{\theta} - \frac{1 - x^i}{1 - \theta} \right) \\ &= \sum_{i=1}^n v^i \frac{x^i(1 - \theta) - (1 - x^i)\theta}{\theta(1 - \theta)} \\ &= \frac{1}{\theta(1 - \theta)} \sum_{i=1}^n v^i (x^i - \theta) \\ &= \frac{1}{\theta(1 - \theta)} \left(\sum_{i=1}^n v^i x^i - \theta \sum_{i=1}^n v^i \right) \\ &= 0 \end{aligned} \tag{2}$$

Thus,

$$\theta_{MLE} = \frac{\sum_{i=1}^n v^i x^i}{\sum_{i=1}^n v^i}$$

2 Multivariate and Conditional Bernoullis

2.1 Naive Bayes

If you run the script `example_mnist35_naiveNaiveBayes`, it will load training and test MNIST digits 3 and 5. It will then discretize the features into binary values, and fit a “naive” naive Bayes model. In particular, the naive naive Bayes model is a generative classifier that assumes $p(x_1, x_2, \dots, x_d, y)$ is product of Bernoullis. This model has a high test error, since the features do not affect the predictions.

1. The regular naive Bayes model discussed in class writes the joint probability of a data $\{X, y\}$ as

$$\begin{aligned} p(X, y) &= \prod_{i=1}^n \left[p(y^i) \prod_{j=1}^d p(x_j^i | y^i) \right] \\ &= \prod_{i=1}^n \left[\theta^{y^i} (1 - \theta)^{1-y^i} \prod_{j=1}^d \theta_{jy}^{x_j^i} (1 - \theta_{jy})^{1-x_j^i} \right], \end{aligned}$$

where we have used a Bernoulli to parameterize $p(y^i)$ and a Bernoulli-like distribution to parameterize the conditionals $p(x_j | y)$. [Show how to derive the MLE for a particular value of \$\theta_{jc}\$.](#)

Answer:

$$\begin{aligned} \log \Pr(X, y) &= \sum_{i=1}^n \log \left[\theta^{y^i} (1 - \theta)^{1-y^i} \prod_{j=1}^d \theta_{jy}^{x_j^i} (1 - \theta_{jy})^{1-x_j^i} \right] \\ &= \sum_{i=1}^n \left[y^i \log \theta + (1 - y^i) \log(1 - \theta) + \sum_{j=1}^d (x_j^i \log \theta_{jy} + (1 - x_j^i) \log(1 - \theta_{jy})) \right] \end{aligned} \quad (3)$$

$$\begin{aligned} \frac{d \log \Pr(X, y)}{d \theta_{jc}} &= \sum_{y^i=c} \left(\frac{x_j^i}{\theta_{jc}} - \frac{1 - x_j^i}{1 - \theta_{jc}} \right) \\ &= \sum_{y^i=c} \frac{x_j^i - \theta_{jc}}{\theta_{jc}(1 - \theta_{jc})} \\ &= 0 \end{aligned} \quad (4)$$

$$\sum_{y^i=c} (x_j^i - \theta_{jc}) = 0$$

$$\sum_{y^i=c} \theta_{jc, MLE} = \sum_{y^i=c} x_j^i$$

Essentially, the MLE estimate is the proportion of examples with $y^i = c$ that have feature j .

2. Even though the naive Bayes likelihood has many parameters, the MLE for a particular parameter θ_{jc} does not depend on θ or any $\theta_{j'c'}$ with $j \neq j'$ and $c' \neq c$. Explain why these terms do not appear in the MLE. (You should not use the word “independence” in your answer. Instead explain how the form of the log-likelihood makes them not depend on each other.)

Answer: The log likelihood turns the original product into a sum of logs. When differentiated with respect to θ_{jc} , the other parameters are constant terms and become 0.

3. Modify the naive naive Bayes code to implement the standard naive Bayes classifier. Hand in your code and report the test error that you obtain. Hint: start from the function *naiveNaiveBayes* and replace the $d \times 1$ variable p_x with a $d \times 2$ variable p_xy that will represent θ_{jc} . Hint: you can look at subsequent questions to get an idea of what the final error should be.

Answer: Obtained Error rate = 0.10

```
include("misc.jl") # Includes GenericModel typedef

function naiveBayes(X,y)
    # Implementation of generative classifier,
    # where a product of Bernoullis is used for p(x,y)

    (n,d) = size(X)
    # Number of each class
    n1 = sum(y.==1)
    n2 = n - n1

    # Compute p(y = 1)
    p_y = sum(y.==1)/n

    # We will store p(x(i,j) = 1|y) in p_xy(j)
    p_xy = zeros(d, 2)
    # Indices of each class
    i1 = findall(==(1), y)
    i2 = findall(!=(1), y)

    for j in 1:d
        p_xy[j, 1] = sum(X[i1, j].==1)/n1
        p_xy[j, 2] = sum(X[i2, j].==1)/n2
    end

    function predict(Xhat)
        (t,d) = size(Xhat)
        yhat = zeros(t)

        for i in 1:t
            # p_yx = p_y*prod(p_x) for the appropriate x and y values
            p_yx = [p_y;1-p_y]
            for j in 1:d
                if Xhat[i,j] == 1
                    p_yx[1] *= p_xy[j,1]
                    p_yx[2] *= p_xy[j,2]
                else
                    p_yx[1] *= 1-p_xy[j,1]
                    p_yx[2] *= 1-p_xy[j,2]
                end
            end
            yhat[i] = argmax(p_yx)
        end
    end
end
```

```
        end
    end
    if p_yx[1] > p_yx[2]
        yhat[i] = 1
    else
        yhat[i] = 2
    end
end
return yhat
end

return GenericModel(predict)
end
```


4. Instead of assuming that all variables are independent given the class label, a less-naive model called “chain-augmented naive Bayes” (CANB) assumes that variables follow a Markov chain given the class label. This leads to a joint likelihood of the form

$$p(x_1^i, x_2^i, \dots, x_d^i, y^i) = p(y^i)p(x_1^i | y^i) \prod_{j=2}^d p(x_j^i | x_{j-1}^i, y^i),$$

where features 2: d depend on not only on the class label but also on the value of the feature that comes “before” it in the ordering 1: d . This captures the fact that adjacent pixels in the image tend to be correlated with each other. Using MLE to estimate the parameters of this model leads to test error of 0.24, while if you add Laplace smoothing it achieves a better test error of 0.14 (which is still worse than naive Bayes). On the other hand, adding Laplace smoothing to naive Bayes does not change the test error for this data. [Explain why Laplace smoothing improves the test error of CANB, while for naive Bayes it does not affect the test error.](#)

[Answer:](#) Laplace smoothing helps against overfitting for CANB, since the “imaginary” data might be able to account for orderings that were not captured for CANB, whereas for naive Bayes, the ordering of the features does not matter, so it would not affect the test error.

5. With Laplace smoothing, the CANB from the previous question obtains a much-higher test-set likelihood than naive Bayes. In other words, $p(\tilde{X}, \tilde{y})$ for new data is much higher for the CANB model than naive Bayes. [Explain why the CANB model might give worse test error even though it gives a much-better test set likelihood.](#)

[Answer:](#) The CANB model may still assign even higher likelihoods to the incorrect classification of test examples, and therefore give a worse test error.

2.2 Vector-Quantized Naive Bayes

If you run the script `example_mnist35_logistic`, it will fit a logistic regression model that achieves a lower test error on the MNIST 3 and 5 digits than naive Bayes does. In this question we will consider one way to make naive Bayes less naive in order to improve its performance.

1. It seems reasonable that there would be clusters in the classes. For example, there might several different ways to draw the digit ‘3’. Instead of assuming that the features are independent given the class label, we might instead assume they are independent given the cluster that they are in. Consider a *vector-quantized naive Bayes* (VQNB) that implements this idea:
 - It clusters the *examples associated with each digit* into k clusters, using k-means clustering (so there will be k clusters for each class and $2k$ clusters total). We will use z^i as the cluster number of example i . The value z^i will be from 1 to k , with y^i determining whether we are considering the k “3” clusters or the k “5” clusters.
 - Since we do not know z^i (it is a “latent” variable), we need to use the marginalization rule to consider its possible values. Using the marginalization and then the product rule, we can write the joint probability as

$$\begin{aligned}
 p(x_1^i, x_2^i, \dots, x_d^i, y^i) &= \sum_{z=1}^k p(x_1^i, x_2^i, \dots, x_d^i, y^i, z) \\
 &= \sum_{z=1}^k p(x_1^i, x_2^i, \dots, x_d^i \mid y^i, z) p(y^i, z) \\
 &= \sum_{z=1}^k p(x_1^i, x_2^i, \dots, x_d^i \mid y^i, z) p(z \mid y^i) p(y^i) \\
 &= p(y^i) \sum_{z=1}^k p(z \mid y^i) \prod_{j=1}^d p(x_j^i \mid y^i, z),
 \end{aligned}$$

where the last line is using the independence of the features given the cluster.

Make a version of your naive Bayes code that implements the VQNB method. [Hand in your code and the test error you obtain with this model for \$k = 2\$ through \$k = 5\$.](#) Hints:

- You can use the variable `p_y` as before. You will need a new variable `p_zy` that is $k \times 2$ representing $p(z = c \mid y^i = b)$. The MLE for an element `p_zy[c,b]` of the matrix is (number of times that $y^i = b$ and $z^i = c$)/(number of times $y^i = b$).²
- You can replace `p_xy` with `p_xyz` which will be a d by 2 by k array giving the value $p(x_j^i = 1 \mid y^i = b, z^i = c)$. The MLE for the value `p_xyz[j,b,c]` is (number of times $y^i = b$ and $z^i = c$ and $x_j^i = 1$)/(number of times $y^i = b$ and $z^i = c$).
- To help with debugging, note that you should get the naive Bayes model in the special case of $k = 1$. Further, with this type of model you usually see the biggest performance gain when going from $k = 1$ to $k = 2$.
- You may not be able to exactly match the performance of logistic regression.

Answer: Obtained error rates:

```

k = 2 : Error rate = 0.07
k = 3 : Error rate = 0.06
k = 4 : Error rate = 0.05
k = 5 : Error rate = 0.05

```

²We will derive this MLE for k -valued discrete variables in the next part of the course.

```

include("misc.jl") # Includes GenericModel typedef
include("kMeans.jl")

function VQNB(X,y,k)
    # Implementation of generative classifier,
    # where a product of Bernoullis is used for p(x,y)
    # in clusters

    (n,d) = size(X)
    # Number of each class
    n1 = sum(y.==1)
    n2 = n - n1

    # Indices of each class
    i1 = findall(==(1), y)
    i2 = findall!=(1), y)

    X1 = X[i1,:]
    X2 = X[i2,:]
    y1 = y[i1]
    y2 = y[i2]

    model1 = kMeans(X1,k)
    model2 = kMeans(X2,k)

    # Clusters for each class
    z1 = model1.predict(X1)
    z2 = model2.predict(X2)

    X12 = [X1; X2]
    y12 = [y1; y2]
    z12 = [z1; z2]

    # Hold num of examples of a cluster for each class
    # nz[c,b] is # yi = b and zi = c
    # nzd[j,b,c] is # yi = b and zi = c and xij = 1
    nz = zeros(k,2)
    nzd = zeros(d,2,k)
    for i in 1:n
        for z in 1:k
            if z12[i] != z
                continue
            end
            nz[z,y12[i]] += 1
            for j in 1:d
                nzd[j,y12[i],z] += X12[i,j] == 1
            end
        end
    end

    # Compute p(y = 1)

```

```

p_y = sum(y.==1)/n

# Compute p(z = c | y_i = b)
p_zy = nz ./ [n1 n2]

# We will store p(x(i,j) = 1|y,z) in p_xyz(j)
p_xyz = zeros(d, 2, k)

for j in 1:d
    p_xyz[j,:,:] = nzd[j,:,:] ./ nz'
end

function predict(Xhat)
    (t,d) = size(Xhat)
    yhat = zeros(t)

    for i in 1:t
        # p_yx = p_y*prod(p_x) for the appropriate x and y values
        p_yx = [p_y;1-p_y]
        p_yzyxyz = [p_y;1-p_y]

        # Sum of p_zy * prod(p_xyz)
        p_zyxyz = zeros(2)
        for z in 1:k
            p_yxyz = p_zy[z,:]
            # Product of p_xyz
            for j in 1:d
                if Xhat[i,j] == 1
                    p_yxyz[1] *= p_xyz[j,1,z]
                    p_yxyz[2] *= p_xyz[j,2,z]
                else
                    p_yxyz[1] *= 1-p_xyz[j,1,z]
                    p_yxyz[2] *= 1-p_xyz[j,2,z]
                end
            end
            p_zyxyz += p_yxyz
        end

        p_yzyxyz .*= p_zyxyz

        if p_yzyxyz[1] > p_yzyxyz[2]
            yhat[i] = 1
        else
            yhat[i] = 2
        end
    end
    return yhat
end

return GenericModel(predict)
end

```

- What is the cost of making a prediction with this model?

Answer:

$$O(tkd)$$

For t training examples, summing over k clusters, each with a product of d features. This assumes only 2 classes.

- For a run of the method with $k = 5$, show the images obtained by plotting the estimates for $p(x_j = 1 \mid z, y)$ for all j as a 28 by 28 image, for each value of z and y (so there should be 10 images). Hint: if you want to view image i with *PyPlot*, you could use `imshow(reshape(X[i, :], 28, 28), 'gray')`.

Answer:



- The logistic regression model and VQNB model obtain similar accuracies, and both give estimates of the probability for $p(y^i \mid x_1^i, x_2^i, \dots, x_d^i)$. Give an example of something we could do with the VQNB model that we could not do with the logistic regression model.

Answer: VQNB gives us information about clusters in the data which logistic regression might not, so we can even do inference based on each cluster within a class.

3 Neural Networks

3.1 Neural Networks by Hand

Suppose that we train a neural network with sigmoid activations and one hidden layer and obtain the following parameters (assume that we don't use any bias variables):

$$W = \begin{bmatrix} -2 & 2 & -1 \\ 1 & -2 & 0 \end{bmatrix}, v = \begin{bmatrix} 3 \\ 1 \end{bmatrix}.$$

Assuming that we are doing regression, for a training example with features $(x^i)^T = [-3 \ -2 \ 2]$ what are the values in this network of the hidden units z^i , activations $a^i = h(z^i)$, and prediction $\hat{y}^i$?

Answer:

$$z^i = Wx^i = \begin{bmatrix} -2(-3) + 2(-2) - 1(2) \\ 1(-3) - 2(-2) \end{bmatrix} = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$$

$$a^i = h(z^i) = \begin{bmatrix} \frac{0.5}{1 + \exp(-1)} \end{bmatrix}$$

$$\hat{y}^i = v^T a^i = 2.231059$$

3.2 Neural Network Tuning - Regression

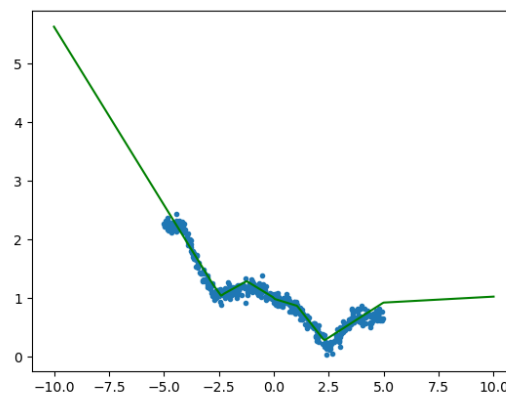
The file `example_nnet.jl` runs a stochastic gradient method to train a neural network on the *basisData* dataset from a previous assignment. However, in its current form it doesn't fit the data very well. Modify the training procedure and model to improve the performance of the neural network. [Hand in your plot after changing the code to have better performance, and list the changes you made.](#)

Hint: there are many possible strategies you could take to improve performance. Below are some suggestions, but note that the some will be more effective than others:

- Changing the network structure (*nHidden* is a vector giving the number of hidden units in each layer).
- Changing the training procedure (you can change the stochastic gradient step-size, use decreasing step sizes, use mini-batches, run it for more iterations, add momentum, switch to *findMin*, use Adam, and so on).
- Transform the data by standardizing the features, standardizing the targets, and so on.
- Add regularization (L2-regularization, L1-regularization, dropout, and so on).
- Change the initialization.
- Add bias variables.
- Change the loss function or the non-linearities (right now it uses squared error and tanh to introduce non-linearity).
- Use mini-batches of data, possibly with batch normalization.

Answer:

- Bias variables were added to all layers.
- Training iteration number increased to 20000.
- Number of hidden units increased to 11.
- Non-linearity changed from tanh to ReLU.
- Increase step size to 0.01.



3.3 Neural Network Tuning - Classification

The file `example_mnist35_nnet.jl` runs a stochastic gradient method to train a neural network on the '3' and '5' images from the naive Bayes question (it uses the squared error for training and classifies by taking the sign of the prediction). Modify the training procedure and model so that the neural network has a better error on the test set than the 0.03 achieved by logistic regression. [List the changes you made and the best test performance that you were able to obtain.](#)

Answer: Achieved **error rate** = 0.02. First 2 changes below similar to 3.2.

- Bias variables were added to all layers.
- ReLU non-linearity used.
- Training iteration number increased to 200000.
- 2 hidden layers used, each with 28 units.
- Sigmoid function was applied to last unit to get $\hat{y}$.
- Initial step-size increased to 0.01.
- L2 Regularization added.

4 Very-Short Answer Questions

1. List 5 different inference tasks you might do with a (single variable) Bernoulli model?

Answer: Given θ ,

- (a) Computing probabilities: $\Pr(x = 0|\theta)$.
 - (b) Computing Dataset probabilities: $\Pr(x^1, \dots, x^n|\theta)$.
 - (c) Decoding: Find x that maximizes $\Pr(x|\theta)$.
 - (d) Decoding Dataset: Find x that maximises $\Pr(x^1, \dots, x^n|\theta)$.
 - (e) Sampling: Generate samples of x according to $\Pr(x|\theta)$ distribution.
2. What is a reason we might want to sample from a distribution?
- Answer: It allows us to make inferences on the population we are interested in.
3. Suppose we have $p(x) \propto \alpha g(x)$ for some continuous variable x , some positive constant α , and some non-negative function g . Give the form of the distribution if we do not use the $\propto$ sign to hide the proportionality constant.

Answer:

$$\Pr(x) = \frac{g(x)}{\int_x g(x)}$$

4. In the CANB model (discussed in a previous question), you would need to estimate $p(x_j^i = 1 \mid x_{j-1}^i, y^i)$ for all values of x_{j-1}^i and y^i . We could consider a generalization of this where we estimate $p(x_j^i = 1 \mid x_{j-1}^i, x_{j-2}^i, \dots, x_{j-k}^i, y^i)$ for some value k (to further relax the conditional independence assumption made by naive Bayes.) Assuming all values are binary we use tabular probabilities, how many parameters would this model need?

Answer:

$$2^{k+1}j$$

2^k values for each x_j , multiplied by 2 classes of y^i .

5. What is the advantage of using a discriminative model for supervised learning, rather than the generative approach of treating supervised learning as special case of density estimation.

Answer: Discriminative models avoid having to model the features, and instead focus on figuring out how the features affect the labels, as opposed to generative classifiers requiring the model of the features and labels.

6. Give a choice of the number of hidden units, the activation function, and likelihood in a neural network with a single hidden layer such that MLE in the model makes the same predictions as the MLE logistic regression model.

Answer: 1 hidden unit, sigmoid activation function, Bernoulli likelihood.

7. Describe an experimental setup where you might see a double descent curve when fitting a logistic regression model with L2-regularization. You can assume you have a way to generate new relevant features. Caution: the global minimum is unique in this model so the double descent would not come from choosing among multiple global minima.

Answer: The initial ascent would correspond to the model overfitting - the subsequent descent could be if we generate new relevant features which may help "regularize" the original features, then train further.

8. Consider a neural network with L hidden layers, where each layer has at most k hidden units and we have c labels in the final layer (and the number of features is d). What is the cost of prediction with this network?

Answer:

$$O(dk + Lk^2 + kc)$$

9. What is one advantage of using automatic differentiation and one disadvantage?

Answer: An advantage is the gradient can be obtained at the same cost as the function, but the disadvantage is that the intermediate calculations need to be stored, requiring a lot of storage.

10. Convolutional networks seem like a pain... why not just use regular (“fully connected”) neural networks for image classification?

Answer: Regular networks would require massive amounts of parameters to be able to perform on images, leading to unfeasible storage costs and overfitting. Convolutions networks take advantage of the sparsity of connections and parameter sharing to avoid those issues.

11. Why do autoencoders have a bottleneck layer?

Answer: The autoencoder can learn a compressed representation of the data from this layer.

12. We discussed multi-label classification using neural networks and a product of Bernoullis model. The product of Bernoullis model assumes the class labels are independent, so why might this model make predictions that seem to capture dependencies between the random variables?

Answer: Using the intersection example from lecture, but with multi-label classification, if intersection 1 and 2 were not independent and are always busy together, we would have $\Pr(x_1) = \Pr(x_2)$ using the product of Bernoullis and could more likely than not predict them to be busy together.

13. Why can fully-convolutional networks segment images of different sizes?

Answer: They use the same convolution parameters across all the segments of images for all layers.